Wi-Fi Proxy Hotspot — Administration & User Guide

Reference build: Raspberry Pi 3B+ (Debian 13 Trixie) — runs on other Debian-based Linux boxes with NetworkManager

1. Overview

This guide covers a Wi-Fi hotspot that sits between any device that joins its wireless network and the internet. Every device that connects to the hotspot has its outbound TCP traffic sent, by a **configurable global default**, either direct or through one of several saved upstream proxy profiles (so those connections appear to originate from that proxy rather than from this network) — while an administrator-chosen internal IP range is always reached directly regardless of that default, and an ordered list of **routing rules** lets specific domains be sent direct or through any saved proxy, independently of the default and of each other.

A small locally-hosted web panel, protected by the box's own Linux login (no separate account to manage), lets an administrator maintain a saved list of upstream proxy profiles — each with its own name, description, type, address and credentials — alongside changing the Wi-Fi network devices connect to, the global default, the internal bypass range, and the routing rules. Every saved profile is reachable at all times; there is no single "active" proxy to switch — the default and each rule simply name which profile (or "direct") they want. All of this happens with no command line required once installed, and the panel also lists which devices are currently connected.

Property	Reference build value
Hardware	Raspberry Pi 3B+ (other Debian-based boxes should work — see Section 3)
OS	Debian 13 (Trixie), NetworkManager + netplan
Wired interface (management / internet)	eth0 — static 192.168.1.10/24, gateway 192.168.1.1
Internal network (bypass range)	192.168.1.0/24
Wireless interface (hotspot)	wlan0 — NetworkManager AP connection "Gateway-AP"
Hotspot subnet	192.168.4.0/24 (gateway 192.168.4.1)
Admin panel	https://192.168.1.10:8443/

The addresses above are examples used consistently throughout this guide; substitute your own internal network's addressing. A typical use is a hotspot in one country whose devices should reach selected services through a proxy in another, while everything else goes direct.

■ **NOTE:** This guide describes both an automated installer (package/install.sh in the project repository) and the full manual procedure it automates. Use the installer for a normal build; read the manual section if you want to understand or adapt what it does.

2. Architecture

Traffic flow for a device joining the hotspot:

- Device associates with the Wi-Fi access point (NetworkManager "shared" connection on the wireless interface) and receives a DHCP lease on the hotspot subnet.
- NetworkManager itself provides DHCP, NAT (masquerade) and IP forwarding for that connection — no separate hostapd/dnsmasq daemons are used.
- An iptables NAT PREROUTING rule intercepts the device's outbound TCP and, for each packet, checks three things in order: is the destination in the internal bypass CIDR (always direct, wins over everything)? does it match a routing rule's ipset (that rule's action — direct, or a named proxy profile — wins)? if neither, the configurable **global default** applies (direct, or a named proxy profile).

• redsocks runs as a single instance with one `redsocks{}` block per saved proxy profile, each listening on its own permanent local port — so every saved profile is reachable simultaneously, and a rule or the default naming a given profile is simply an iptables REDIRECT to that profile's port. When the outcome is "direct", the packet is simply returned to normal routing/NAT instead, exactly like internal-network traffic.

• Non-TCP traffic (e.g. DNS) is not intercepted by redsocks and follows normal routing/NAT — see the Notes section for the privacy implication of this.

■ **NOTE:** A single Wi-Fi radio cannot normally act as a hotspot and a client of another network at the same time. If the wireless interface was previously connected to an existing Wi-Fi network, building this hotspot on it will end that connection permanently (or use a second, dedicated Wi-Fi adapter for the hotspot if that existing connection must be kept).

2.1 — How routing rules work

redsocks and iptables only see IP addresses and ports — they have no visibility into the hostname a device is actually trying to reach. To route by domain, a small daemon (**dns_router.py**, running as its own unprivileged service account, granted only the CAP_NET_ADMIN capability via systemd) sits between the hotspot's own dnsmasq and the real internet:

• Every DNS query from a hotspot device reaches dnsmasq as normal, which forwards it to the daemon as its first-choice upstream (dnsmasq runs in strict order; the box's own resolvers are only a fallback if the daemon doesn't answer, so DNS keeps working, just without tagging, if the daemon is ever down).

• dnsmasq answers repeat lookups from its own cache without asking the daemon, so two things keep rule changes from waiting on that cache: every firewall rebuild (which every rule change triggers) clears dnsmasq's cache, and dnsmasq hands clients a TTL of at most 60 seconds (max-ttl=60), so devices ask again within a minute. A new, edited, enabled or disabled rule therefore reaches already-connected devices within about a minute for new connections; a connection that is already open (a video mid-playback) keeps its route until it reconnects.

• The daemon checks the queried hostname against the admin's ordered list of pattern rules (exact match, wildcard, or regular expression) — the first match wins.

• The daemon forwards the query to the box's real upstream DNS servers and always returns the genuine answer — it never blocks, delays meaningfully, or rewrites a DNS response.

• If a rule matched, the resolved IP address(es) are tagged into that rule's own `ipset` (with a 24-hour timeout, so stale entries self-expire). An IP the daemon hasn't tagged in the last hour is tagged *before* the answer is returned (one `ipset restore` call, ~4ms), so the device's very first connection to it is already routed correctly; repeat lookups add no delay. Changing a rule's pattern or mode gives it a fresh, empty ipset, so addresses its old pattern matched don't stay routed by it.

• Proxied Lookups (on by default, Routing Rules page): a hostname matched by a rule that routes through a proxy is looked up *through that proxy*, over DNS-over-HTTPS to Google (dns.google), so CDNs answer with servers near the proxy instead of near the hotspot. Measured on the reference build, with the hotspot and the proxy on different continents, streaming a live video channel: a 10-second segment took a median 7.3 s from the CDN server near the hotspot (worst 10.2 s, longer than it plays) vs 3.7 s from the one near the proxy that it now gets. EDNS Client Subnet would not have helped: the CDN in question (Akamai) ignores it. The daemon keeps two tunnels open through each such proxy and pings them to keep them warm, so these lookups cost one round trip (~0.3 s). If no tunnel is ready it uses ordinary DNS at once, never waiting. The default route, and everything not matched by a proxied rule, keep ordinary DNS.

• Separately, for the admin panel's DNS Lookups page, the daemon also passively watches the DNS queries devices send on the hotspot interface (a kernel packet filter lets through only UDP to port 53, so ordinary traffic never reaches it; this needs the CAP_NET_RAW capability as well). That is the only place a lookup can be tied to the device that made it: every query reaching the daemon itself comes from dnsmasq, and names dnsmasq answers from cache never reach it at all. The record is in memory only, snapshotted to /run/rasp-proxy-dns/lookups.json (RAM, readable only by the service group). If capture ever fails, only that page is affected, never DNS routing.

• The firewall's REDSOCKS chain has one rule per pattern rule, in the same order, matching traffic against that rule's ipset and sending it direct or into the named profile's redsocks port accordingly — exactly like the internal bypass CIDR, just IP-set-based instead of a static range. After all pattern rules, one final rule applies the configurable global default to anything left unmatched. An action naming a profile that has since been deleted is treated as direct (fail-safe) rather than erroring the whole rebuild — though the panel refuses to delete a profile that is still referenced anywhere, so this should only happen from manual JSON edits.

■ **NOTE:** This only catches domains actually resolved through the hotspot's own DNS server. The Leak Guards (Routing Rules page, all on by default) close the common gaps: plain DNS sent to an outside server is redirected to the hotspot, DNS-over-TLS (port 853) is blocked, Firefox's DoH is switched off via its canary domain, iCloud Private Relay is turned off for the network, and forwarded IPv6 is blocked. What remains is an app with its own hard-coded DNS-over-HTTPS (indistinguishable from ordinary HTTPS on port 443). Such an app still gets whichever default is configured, so "direct by default, proxy only for specific domains" is not a hard guarantee for every possible packet.

■ **NOTE:** Because matching is by resolved IP rather than by inspecting each connection's real destination, a large CDN-hosted domain that shares IP addresses with unrelated sites can occasionally over-match (an unrelated site on the same IP gets the same treatment) or under-match (a domain's IP changes before the next lookup refreshes the ipset). Keep this in mind for domains fronted by big shared CDNs; it is rarely noticeable for single-tenant services.

■ **WARNING:** The DNS router is a new always-running service directly in the hotspot's DNS resolution path. It is designed to fail open (any internal error just forwards the query unmatched rather than blocking it) and restarts automatically on crash, but if it stops entirely, hotspot devices lose DNS resolution, not just pattern matching. Its own health is worth including in routine checks (see Section 5).

3. Prerequisites

3.1 — Hardware and operating system

The project was built and tested on a **Raspberry Pi 3B+ running Debian 13 (Trixie) / Raspberry Pi OS**, and that is the only hardware it has actually been tested on. Nothing in it is Pi-specific, though, apart from the optional under-voltage and temperature readings on the Overview page (which simply show as unavailable elsewhere), so any Debian 12+ based Linux box should work — for example a Raspberry Pi 4 or 5, a Raspberry Pi Zero 2 W with a USB Ethernet adapter, or a small x86 mini PC — provided it has:

- NetworkManager managing its network interfaces (the `nmcli` command must exist). Older Raspberry Pi OS Legacy/Bullseye images use `dhcpcd`/`hostapd`/`dnsmasq` instead — see the note in Section 8 for the manual equivalent on that stack.
- A wired Ethernet interface for the upstream internet connection and for management (the admin panel is only served there).
- A Wi-Fi adapter that supports access-point mode: run `iw list` and check that "Supported interface modes" includes "AP".
- About 1 GB of RAM is plenty; the whole stack is light enough for a Pi 3B+.

```
iw list | grep -A 10 "Supported interface modes"   # must include: * AP
```

3.2 — Set the Wi-Fi country (regulatory domain) first

The Wi-Fi radio's usable channels depend on the country it is set to, and on Raspberry Pi OS the radio stays blocked (rfkill) until a country has been set at all — the hotspot then simply fails to start. Set it before installing, replacing XX with your two-letter country code (e.g. GB, US, DE):

```
# Raspberry Pi OS (or interactively: sudo raspi-config -> Localisation Options -> WLAN Country)
sudo raspi-config nonint do_wifi_country XX

# Other distributions: set it now ...
sudo iw reg set XX
# ... and persist it the distribution's way, e.g. by adding
#     cfg80211.ieee80211_regdom=XX
# to the kernel command line

# Either way, check the radio isn't blocked, and unblock it if it is:
rfkill list
sudo rfkill unblock wifi
iw reg get   # confirm the country took effect
```

■ **NOTE:** The admin panel's band and channel picker only offers channels the radio reports as usable for an access point under the current country. If no 5 GHz choices appear, the Wi-Fi country is most likely not set.

3.3 — What you need before installing

- A wired interface already reachable on your internal network (DHCP is fine — it becomes static during setup).
- A wireless interface free to dedicate to the hotspot.
- Root access on the box, and `git` to fetch the code.
- The upstream proxy's type (HTTP or SOCKS5), address, port, and credentials if it requires authentication.
- The internal IP range (CIDR) that hotspot devices should be able to reach directly, without going through the proxy.
- Any domains you already know should always (or never) go through the proxy, if using routing rules from the start — these can also be added later via the panel.

4. Installation (Admin)

4.1 — Quick install (recommended)

Clone the repository onto the target machine and run the installer as root from its **package/** folder. It interactively asks for every value in the Prerequisites section above and applies all of Section 4.2 automatically.

```
sudo apt-get install -y git
git clone https://github.com/dassige/wifi-proxy-hotspot.git
cd wifi-proxy-hotspot/package && sudo ./install.sh
```

■ **NOTE:** Run the installer from a full clone rather than copying the package/ folder on its own: it reads the version number from pyproject.toml, one directory above package/, and stops if it is missing. Besides the packages listed in Step 2 it makes sure dnsmasq-base (the DNS/DHCP server NetworkManager uses for the hotspot) and openssl (for the panel's certificate) are installed, since minimal images don't always include them.

■ **WARNING:** If you are connected to the box over SSH via the same wired interface being given a new static IP, your session will drop partway through the install — this is expected. Wait a few seconds and reconnect on the new address.

At the end, the installer prints the hotspot SSID/passphrase and the admin panel URL. To remove everything it created, run **sudo ./uninstall.sh** from the same folder.

4.2 — Manual install, step by step

This is exactly what install.sh automates, for reference or manual adaptation.

Step 1 — Static IP on the wired interface

Find the active NetworkManager connection on the interface and set it to a fixed address:

```
CONN=$(nmcli -t -f GENERAL.CONNECTION device show eth0 | cut -d: -f2)
sudo nmcli connection modify "$CONN" \
    ipv4.method manual ipv4.addresses 192.168.1.10/24 \
    ipv4.gateway 192.168.1.1 ipv4.dns "192.168.1.1 1.1.1.1"
sudo nmcli connection up "$CONN"
```

■ **WARNING:** If the interface already has two NetworkManager profiles (e.g. an auto-created default alongside a netplan-managed one), only one can be active at a time and the wrong one winning is a common, silent failure mode. Confirm with `nmcli connection show` that the DEVICE column points the profile you just modified at the interface, and disable autoconnect on any other profile bound to the same interface: `nmcli connection modify <other-name> connection.autoconnect no`.

Step 2 — Install packages

```
echo iptables-persistent iptables-persistent/autosave_v4 boolean false | sudo debconf-set-selections
echo iptables-persistent iptables-persistent/autosave_v6 boolean false | sudo debconf-set-selections
sudo apt-get update
sudo DEBIAN_FRONTEND=noninteractive apt-get install -y \
    redsocks iptables-persistent python3-flask python3-pam gunicorn iw \
    python3-dnspython ipset python3-qrcode dnsmasq-base openssl
```

Step 3 — Create the Wi-Fi access point

NetworkManager's own "shared" method provides DHCP, NAT and IP forwarding for this connection — no hostapd/dnsmasq needed. The dnsmasq-shared.d file points the hotspot's DNS at the pattern-router daemon (deployed in Step 7, started in Step 7b) and caps the TTL handed to clients at 60s so rule changes reach them quickly; it must exist before the first activation (NetworkManager only reads it when it starts the hotspot's dnsmasq):

```
sudo nmcli connection add type wifi ifname wlan0 con-name Gateway-AP \
    autoconnect yes ssid "<your-ssid>"
sudo nmcli connection modify Gateway-AP 802-11-wireless.mode ap 802-11-wireless.band bg
sudo nmcli connection modify Gateway-AP wifi-sec.key-mgmt wpa-psk
sudo nmcli connection modify Gateway-AP wifi-sec.psk "<your-passphrase>"
sudo nmcli connection modify Gateway-AP ipv4.method shared ipv4.addresses 192.168.4.1/24

sudo mkdir -p /etc/NetworkManager/dnsmasq-shared.d
sudo tee /etc/NetworkManager/dnsmasq-shared.d/pattern-routing.conf > /dev/null <<EOF
server=127.0.0.1#15353
max-ttl=60
EOF

sudo nmcli connection up Gateway-AP
```

Step 4 — Configure redsocks

Write /etc/redsocks.conf with one redsocks{} block per saved proxy profile, each on its own permanent local port starting at 20000 (this is what lets different rules use different proxies simultaneously) — the first profile, created here, always takes 20000:

```
sudo tee /etc/redsocks.conf > /dev/null <<EOF
base {
    log_info = on;
    daemon = on;
    redirector = iptables;
}
redsocks {
    local_ip = 0.0.0.0;
    local_port = 20000;
    ip = <proxy-ip>;
    port = <proxy-port>;
    type = http-connect;   # or socks5
    login = "<proxy-username>";
    password = "<proxy-password>";
}
EOF
sudo chown root:redsocks /etc/redsocks.conf && sudo chmod 660 /etc/redsocks.conf
sudo systemctl enable --now redsocks
```

Step 4b — redsocks performance patches (optional)

Stock redsocks waits for the proxy to reject a connection before sending its Basic-auth credentials, and never sets TCP_NODELAY on its sockets; both cost extra round trips on a long, high-latency link to a proxy abroad. package/redsocks-patches/ holds two small patches fixing that. The installer fetches the distribution's own redsocks source with `apt-get source redsocks` (if no deb-src entry is configured it adds a temporary one for the running release, using the codename from /etc/os-release, and removes it afterwards), applies both patches to whatever source version apt fetched, builds it and replaces /usr/sbin/redsocks, keeping the stock binary as /usr/sbin/redsocks.orig-<version>. This step is best-effort: if anything fails the stock binary stays in place and the install carries on. To do it by hand:

```
sudo apt-get install -y build-essential dpkg-dev libevent-dev
mkdir -p /tmp/redsocks-build && cd /tmp/redsocks-build
apt-get source redsocks          # needs a deb-src entry for your release
cd redsocks-*/
patch -p1 < /path/to/package/redsocks-patches/0001-tcp-nodelay.patch
patch -p1 < /path/to/package/redsocks-patches/0002-preemptive-basic-auth.patch
make
sudo cp /usr/sbin/redsocks /usr/sbin/redsocks.orig
sudo install -m 755 -o root -g root redsocks /usr/sbin/redsocks
sudo systemctl restart redsocks
```

> ■ **NOTE:** A later redsocks package upgrade puts the stock binary back. Re-run the steps above (or re-run the installer) to reapply the patches; everything works either way, only more slowly through a distant proxy without them.

Step 5 — Firewall: redirect hotspot traffic into redsocks

This is the baseline chain matching the one profile created so far — internal range excluded (always direct), everything else redirected to that profile's port 20000. Step 7a rebuilds this properly from the actual JSON stores once they exist, using the same helper the admin panel uses for every later change — this manual version only needs to get redsocks working long enough to reach that point:

```
sudo iptables -t nat -N REDSOCKS
sudo iptables -t nat -A REDSOCKS -d 192.168.1.0/24 -j RETURN
sudo iptables -t nat -A REDSOCKS -p tcp -j REDIRECT --to-ports 20000
sudo iptables -t nat -A PREROUTING -i wlan0 -p tcp -j REDSOCKS
sudo netfilter-persistent save
```

■ **WARNING:** A REDIRECT rule's --to-ports option requires an explicit port-capable protocol match (-p tcp) on that specific rule — it is not inherited from the chain it lives in. Omitting it fails with "Need TCP, UDP, SCTP or DCCP with port specification". This matters for every rule that targets REDIRECT: the catch-all above when the default names a proxy profile, and any pattern rule whose action names a profile. A rule targeting RETURN (default "direct", or a pattern rule set to "direct") needs no protocol match at all.

Step 6 — Dedicated, unprivileged service accounts

One for the admin panel, one for the DNS pattern-router daemon (Step 7b):

```
sudo useradd --system --no-create-home --shell /usr/sbin/nologin --gid redsocks proxyadmin-svc
sudo useradd --system --no-create-home --shell /usr/sbin/nologin --gid redsocks rasp-proxy-dns
```

Step 7 — Deploy the admin web app and its root-owned helpers

From the package folder:

```
sudo mkdir -p /opt/rasp-proxy-admin
sudo install -m 640 -o root -g redsocks app.py /opt/rasp-proxy-admin/app.py
sudo install -m 640 -o root -g redsocks dns_router.py /opt/rasp-proxy-admin/dns_router.py
sudo install -m 640 -o root -g redsocks pattern_match.py /opt/rasp-proxy-admin/pattern_match.py
sudo install -m 640 -o root -g redsocks proxy_check.py /opt/rasp-proxy-admin/proxy_check.py
sudo install -m 640 -o root -g redsocks proxy_doh.py /opt/rasp-proxy-admin/proxy_doh.py
sudo install -m 640 -o root -g redsocks mailer.py /opt/rasp-proxy-admin/mailer.py
sudo install -m 640 -o root -g redsocks help_texts.py /opt/rasp-proxy-admin/help_texts.py
sudo cp -r templates /opt/rasp-proxy-admin/ && sudo chown -R root:redsocks /opt/rasp-proxy-admin/templates
sudo find /opt/rasp-proxy-admin/templates -type d -exec chmod 750 {} +
sudo find /opt/rasp-proxy-admin/templates -type f -exec chmod 640 {} +
sudo chown root:redsocks /opt/rasp-proxy-admin && sudo chmod 750 /opt/rasp-proxy-admin
sudo install -m 750 -o root -g root rasp-proxy-pamcheck.py /usr/local/sbin/rasp-proxy-pamcheck
sudo install -m 750 -o root -g root rasp-proxy-rebuild-firewall.py /usr/local/sbin/rasp-proxy-rebuild-fir
sudo install -m 750 -o root -g root rasp-proxy-traffic-sync.py /usr/local/sbin/rasp-proxy-traffic-sync
sudo install -m 750 -o root -g root rasp-proxy-health.py /usr/local/sbin/rasp-proxy-health
sudo install -m 644 -o root -g root pam-service /etc/pam.d/rasp-proxy-admin
sudo mkdir -p /etc/rasp-proxy-admin/tls
sudo openssl req -x509 -nodes -newkey rsa:2048 \
    -keyout /etc/rasp-proxy-admin/tls/key.pem -out /etc/rasp-proxy-admin/tls/cert.pem \
    -days 3650 -subj "/CN=wifi-proxy-hotspot.local"
sudo chown -R root:redsocks /etc/rasp-proxy-admin
sudo chmod 750 /etc/rasp-proxy-admin /etc/rasp-proxy-admin/tls
sudo chmod 640 /etc/rasp-proxy-admin/tls/*.pem
```

Step 7a — Create the mutable state directory and stores

Both the proxy-profile list and the pattern-rule list live under /var/lib, not /etc, and must be owned by the service account itself — /var/lib is not group-writable, so the unprivileged app could never create this directory on its own the first time it tries to save anything:

```
sudo mkdir -p /var/lib/rasp-proxy-admin
sudo chown proxyadmin-svc:redsocks /var/lib/rasp-proxy-admin
sudo chmod 750 /var/lib/rasp-proxy-admin

# bypass_cidr lives in this file (not derived from live iptables output)
# specifically so the firewall can be rebuilt from scratch at boot — see
# Step 11 for why that matters:
sudo tee /var/lib/rasp-proxy-admin/pattern_rules.json > /dev/null <<EOF
{
  "bypass_cidr": "192.168.1.0/24",
  "default_action": "default",
  "rules": []
}
EOF
sudo chown proxyadmin-svc:redsocks /var/lib/rasp-proxy-admin/pattern_rules.json
sudo chmod 640 /var/lib/rasp-proxy-admin/pattern_rules.json

# Re-derive the chain from these stores using the same helper the panel
# uses for every later change, rather than trusting Step 5's hand-built
# version to stay in sync with it:
sudo /usr/local/sbin/rasp-proxy-rebuild-firewall wlan0
```

■ **WARNING:** This step is not optional. The panel auto-creates a "default" proxy profile (assigned local port 20000, matching Step 4) from whatever is in /etc/redsocks.conf the first time it loads — but that first write still needs somewhere to land. Skip this step and the very first page load will fail outright, since the unprivileged app cannot create a new directory under /var/lib itself. The default_action value above assumes that auto-created profile is still named "default"; if you already know you'll rename or replace it, adjust accordingly once the panel is up.

Step 7b — DNS pattern-router service

Runs as the unprivileged rasp-proxy-dns account, with only CAP_NET_ADMIN granted via systemd — enough to call `ipset` directly, nothing else:

```
sudo tee /etc/systemd/system/rasp-proxy-dns-router.service > /dev/null <<EOF
[Unit]
Description=Wi-Fi Proxy Hotspot DNS pattern router
After=network-online.target
[Service]
Type=simple
User=rasp-proxy-dns
Group=redsocks
AmbientCapabilities=CAP_NET_ADMIN CAP_NET_RAW
CapabilityBoundingSet=CAP_NET_ADMIN CAP_NET_RAW
NoNewPrivileges=true
Environment=RASP_PROXY_AP_IFACE=wlan0
RuntimeDirectory=rasp-proxy-dns
RuntimeDirectoryMode=0750
WorkingDirectory=/opt/rasp-proxy-admin
ExecStart=/usr/bin/python3 /opt/rasp-proxy-admin/dns_router.py
Restart=always
RestartSec=2
[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload && sudo systemctl enable --now rasp-proxy-dns-router
```

Step 8 — Sudoers: the exact, minimal root actions the web app needs

Validate with `visudo -c -f` before installing to /etc/sudoers.d/ — a syntax error here can lock out sudo entirely:

```
proxyadmin-svc ALL=(root) NOPASSWD: /usr/local/sbin/rasp-proxy-pamcheck admin
proxyadmin-svc ALL=(root) NOPASSWD: /usr/bin/systemctl restart redsocks
proxyadmin-svc ALL=(root) NOPASSWD: /usr/bin/nmcli connection modify Gateway-AP *
proxyadmin-svc ALL=(root) NOPASSWD: /usr/bin/nmcli connection up Gateway-AP
proxyadmin-svc ALL=(root) NOPASSWD: /usr/bin/nmcli connection down Gateway-AP
proxyadmin-svc ALL=(root) NOPASSWD: /bin/cat /var/lib/NetworkManager/dnsmasq-wlan0.leases
proxyadmin-svc ALL=(root) NOPASSWD: /usr/local/sbin/rasp-proxy-rebuild-firewall *
proxyadmin-svc ALL=(root) NOPASSWD: /usr/sbin/iptables -t nat -L REDSOCKS -n
proxyadmin-svc ALL=(root) NOPASSWD: /usr/bin/systemd-run --on-active=3 /usr/sbin/reboot
proxyadmin-svc ALL=(root) NOPASSWD: /usr/local/sbin/rasp-proxy-traffic-sync *
proxyadmin-svc ALL=(root) NOPASSWD: /usr/sbin/iptables -t mangle -L WPXACCT_UP -n -v -x
proxyadmin-svc ALL=(root) NOPASSWD: /usr/sbin/iptables -t mangle -L WPXACCT_DOWN -n -v -x
proxyadmin-svc ALL=(root) NOPASSWD: /usr/bin/systemctl start --no-block rasp-proxy-health.service
proxyadmin-svc ALL=(root) NOPASSWD: /usr/bin/nmcli -s -g 802-11-wireless-security.psk connection show Gat
```

Step 9 — systemd service for the admin panel

```
sudo tee /etc/systemd/system/rasp-proxy-admin.service > /dev/null <<EOF
[Unit]
Description=Wi-Fi Proxy Hotspot admin web panel
After=network-online.target
[Service]
Type=simple
User=proxyadmin-svc
Group=redsocks
WorkingDirectory=/opt/rasp-proxy-admin
Environment=RASP_PROXY_ADMIN_USERS=admin
Environment=RASP_PROXY_AP_CONN=Gateway-AP
Environment=RASP_PROXY_AP_IFACE=wlan0
Environment=RASP_PROXY_VERSION=1.22.0
Environment="RASP_PROXY_VERSION_DATE=<install date, YYYY/MM/DD HH:MM>"
ExecStart=/usr/bin/gunicorn --bind 192.168.1.10:8443 \
  --certfile=/etc/rasp-proxy-admin/tls/cert.pem --keyfile=/etc/rasp-proxy-admin/tls/key.pem \
  --workers 2 app:app
Restart=on-failure
[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload && sudo systemctl enable --now rasp-proxy-admin
```

■ **NOTE:** Every service and connection above is enabled/autoconnect by default, so the whole stack — static IP, hotspot, redsocks, and the admin panel — comes back automatically after a reboot with no further action, confirmed by an actual reboot on the reference build. The firewall chain itself needs the dedicated boot service in Step 11 below to survive a reboot correctly — see the warning there for why.

Step 11 — Boot-time firewall rebuild (do not skip this)

`netfilter-persistent` saves iptables rules across reboots, but never `ipset` contents. Since the REDSOCKS chain's pattern-rule entries reference ipsets by name, a saved rule pointing at an ipset that no longer exists after reboot makes the restore of the *entire* saved ruleset fail — `iptables-restore` applies its input as one atomic transaction. The practical effect discovered on the reference build: after a reboot, the REDSOCKS chain did not exist at all, and hotspot traffic went out completely unfiltered — no proxy, no pattern rules, silently. This service rebuilds the whole chain and its ipsets from scratch, from the same JSON state the admin panel uses, every time the box starts:

```
sudo tee /etc/systemd/system/rasp-proxy-firewall-init.service > /dev/null <<EOF
[Unit]
Description=Wi-Fi Proxy Hotspot firewall init (rebuilds REDSOCKS chain + ipsets on boot)
After=network-online.target redsocks.service
Wants=network-online.target
Requires=redsocks.service
[Service]
Type=oneshot
ExecStart=/usr/local/sbin/rasp-proxy-rebuild-firewall wlan0
RemainAfterExit=yes
[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload && sudo systemctl enable --now rasp-proxy-firewall-init
```

■ **WARNING:** If this service is missing (e.g. on a box built before it existed), the REDSOCKS chain will silently fail to come back after every reboot, and hotspot traffic will go out completely un-proxied and un-filtered with no error shown anywhere — check for it explicitly (`systemctl is-enabled rasp-proxy-firewall-init`) if migrating an older build.

Step 12 — Health check timer (Overview page + email alerts)

A root oneshot, run every 2 minutes by a timer, that checks the services, hotspot, firewall chains, DNS, every proxy and the box itself, and writes the results for the panel's Overview page. It also sends the email alerts configured on the System page. It only ever reports; it never changes routing. Root is needed to read the firewall chains and the kernel log. Use the box's real panel address in RASP_PROXY_PANEL_URL (it goes into alert emails):

```
sudo tee /etc/systemd/system/rasp-proxy-health.service > /dev/null <<EOF
[Unit]
Description=Wi-Fi Proxy Hotspot health check (Overview page + email alerts)
After=network-online.target
[Service]
Type=oneshot
Environment=RASP_PROXY_AP_IFACE=wlan0
Environment=RASP_PROXY_AP_CONN=Gateway-AP
Environment=RASP_PROXY_PANEL_URL=https://192.168.1.10:8443/
ExecStart=/usr/local/sbin/rasp-proxy-health
TimeoutStartSec=120
Nice=10
EOF
sudo tee /etc/systemd/system/rasp-proxy-health.timer > /dev/null <<EOF
[Unit]
Description=Run the Wi-Fi Proxy Hotspot health check every 2 minutes
[Timer]
OnBootSec=90s
OnUnitActiveSec=2min
AccuracySec=15s
[Install]
WantedBy=timers.target
EOF
sudo systemctl daemon-reload && sudo systemctl enable --now rasp-proxy-health.timer
```

5. Admin — Day-to-Day Operation

Once installed, routine changes go through the web panel rather than the command line: open **https://<wired-ip>:8443/** and log in with a Linux account listed in RASP_PROXY_ADMIN_USERS on the box. The browser will warn about the self-signed certificate; that is expected — accept it. The panel is a left-hand menu (Overview, Proxy Profiles, Wi-Fi Access Point, Internal Network, Routing Rules, Devices, DNS Lookups, Backup & Restore, System, Help) with one page shown at a time, rather than one long scrolling form — the browser remembers which page you were on across actions, and following an Edit link opens straight to the right page. The panel also works from a phone: on a narrow screen the menu becomes a row of tabs across the top that scrolls sideways, and each row of the wider tables (Proxy Profiles, Routing Rules, Devices) is shown as its own labelled block instead of a table row. The panel is only reachable on the wired side, so the phone must be on that network, not on the hotspot.

Task	Where
See whether everything is working, at a glance	"Overview" page (the first page; a coloured dot next to it in the menu shows
Get an email when something breaks, and when it recovers	"System" → "Email Alerts" — SMTP server, port, security, username/passw
Add a new proxy profile (name, description, type, address, port, credentials)	"Proxy Profiles" → "Add Proxy Profile" (collapsed by default; click to expand

Edit an existing proxy profile	"Proxy Profiles" table → Edit button — opens the same form pre-filled; leave
Disable a proxy you only run now and then (e.g. a test proxy you start on demand)	"Proxy Profiles" table → its "Enabled" toggle — a disabled profile isn't health
Remove a saved proxy profile	"Proxy Profiles" table → Delete button (confirmation required) — refused wit
Change the hotspot SSID or passphrase	"Wi-Fi Access Point" page — reactivates the hotspot (if currently enabled); c
Move the hotspot to 5 GHz (or another channel)	"Wi-Fi Access Point" → "Band & channel" → Apply (confirmation required) –
Let someone join by scanning a code instead of typing the password	"Wi-Fi Access Point" → "Show join QR code" — a standard Wi-Fi QR code p
Switch the panel to dark mode	The "Theme" picker at the bottom of the left-hand menu — Auto (follows the
Restart the Wi-Fi access point without changing anything	"Wi-Fi Access Point" → "Restart Wi-Fi" button (confirmation required, disabl
Turn the hotspot radio off entirely, or back on	"Wi-Fi Access Point" → "Disable Hotspot"/"Enable Hotspot" button (confirma
Change which internal range bypasses the proxy	"Internal Network" page — rewrites and persists the firewall rule
Change the global default (direct, or a named proxy profile) for everything not otherwise matched	"Routing Rules" page, "Default Behaviour" section at the top — rebuilds and
Add a routing rule (route a domain direct or to any saved proxy)	"Routing Rules" → "Add Routing Rule" (collapsed by default) — choose exa
Cover a whole service with one rule (site + its video CDNs)	"Routing Rules" → Add/Edit — the Patterns box takes one pattern per line, a
Check what will happen to a hostname before (or instead of) trying it on a device	"Routing Rules" → "Test a hostname" — type a hostname or paste a URL: s
See whether each rule is actually doing anything	"Routing Rules" table → Activity column — recent lookups each rule matche
Stop devices routing around the rules	"Routing Rules" → "Leak Guards" (all on by default): <b>Force all DNS thro
Make proxied video fetch from servers near the proxy, not near the hotspot	"Routing Rules" → "Proxied Lookups" toggle (on by default) — hostnames r
Reorder pattern rules (earlier rules win)	"Routing Rules" table → Move Up / Move Down on each rule's action row
Temporarily switch a pattern rule off without deleting it	"Routing Rules" table → the toggle switch in that rule's Status column — a c
Edit or remove a pattern rule	"Routing Rules" table → Edit or Delete (confirmation required) on that rule's
See every device that has ever joined, not just those currently online	"Devices" page — hostname, IP, MAC, Online/Offline/Blocked status, conne
Give a device a recognisable name	"Devices" table → "name it" / "rename" under the device — the nickname is
Make one device always go direct, or always through a particular proxy	"Devices" table → Routing — Follow the rules (default), Always direct, or Alw
Block a device from the hotspot, or unblock it	"Devices" table → the toggle switch in that device's Blocked column (confirm
Back up the current configuration	"Backup & Restore" → "Export configuration" — downloads everything confi
Restore or clone a configuration	"Backup & Restore" → choose a previously exported file, "Import configurati
Turn a disabled proxy back on, with what it switched off	"Proxy Profiles" → its Enabled toggle — asks whether to switch back on the
Reboot the whole box (not just the Wi-Fi radio)	"System" → "Reboot Server" button (confirmation required) — the panel and
See how much traffic each connected device has sent/received	"Devices" table — Sent/Received columns update automatically every few s
Find out which hostnames a site, app or video really uses (to write the right rules)	"DNS Lookups" page (or the "lookups" link next to a device on the Devices p
Check how strong each connected device's Wi-Fi signal is	"Devices" table → Signal column — signal bars plus the reading in dBm (clo
Open this guide from the panel itself, or check which version is running	"Help" page → "Open the Wi-Fi Proxy Hotspot Guide (PDF)" — opens this P
See the credits, license and disclaimer	"Help" page — the author (Gerry Dassi), the open-source projects it is built o

Every section above also has a small (?) help icon opening a short in-panel explanation with examples — the Routing Rules one covers all three pattern modes in detail, mirroring Section 2.1 of this guide.

Every saved profile is always reachable; the "Default Behaviour" section and each pattern rule's Action dropdown are the only places that determine which traffic actually uses which profile at a glance.

The per-device traffic counters are deliberately built on plain iptables byte counters rather than any packet-capture or per-connection logging approach, specifically because a Pi 3B+ has little CPU to spare: each connected device gets exactly one counting rule in each of two `mangle`-table chains (upload, hooked from `PREROUTING`; download, hooked from `POSTROUTING`), and the kernel is already touching every packet these chains see regardless — the counters add a rule-table lookup and an integer increment, not new work. The admin panel reads those counters (a single `iptables -L` call per direction) roughly every 5 seconds while the page is open; it does not run a background collector, so closing the browser tab stops the polling entirely. Counting happens before any REDSOCKS redirect decision (upload) or from wherever the reply actually originates (download), so direct and proxied traffic are both counted correctly without needing to know which path a given connection took. These counters are intentionally not wired into `netfilter-persistent` and do not survive a reboot — unlike the REDSOCKS chain (Step 11), that is the right choice here: they are live session statistics, rebuilt fresh and cheaply by the admin panel itself the next time it renders, not policy state anything else depends on.

Enabling/disabling the hotspot and blocking/unblocking a device both build on the same NetworkManager mechanics already used for SSID/passphrase changes: "Disable Hotspot" sets the `Gateway-AP` connection's `connection.autoconnect` to no and brings it down, so it stays off across a reboot until explicitly re-enabled, rather than just being down until the next boot; "Enable Hotspot" reverses both. Blocking a device adds its MAC address to that same connection's `802-11-wireless.mac-address-denylist` property (the standard NetworkManager equivalent of hostapd's MAC ACL) and reactivates the connection — reactivation is necessary both to force-disconnect a device that's already associated and to make an unblock take effect, the same reason SSID/passphrase changes reactivate. Every known device (from `devices.json`, keyed by MAC) is shown on the Devices page regardless of whether it is currently online, specifically so a blocked device stays visible and can be unblocked later even though it can no longer reconnect to show up as "live."

Useful command-line checks if something needs deeper investigation:

```
sudo systemctl status redsocks rasp-proxy-admin rasp-proxy-dns-router
sudo journalctl -u rasp-proxy-admin --since "1 hour ago"
sudo journalctl -u rasp-proxy-dns-router --since "1 hour ago"
sudo iptables -t nat -L REDSOCKS -n -v
sudo ipset list -n
nmcli device status
sudo iw dev wlan0 station dump
nmcli -t -f NAME connection show --active          # is Gateway-AP currently enabled?
nmcli -g 802-11-wireless.mac-address-denylist connection show Gateway-AP   # currently blocked MACs
```

6. User Guide — Connecting a Device

For anyone connecting a phone, laptop, or other device to this hotspot:

- Join the Wi-Fi network using the SSID and passphrase given to you by the administrator, exactly as you would any other Wi-Fi network.
- Once connected, your device works normally — web browsing, apps, streaming — with no extra setup on your end.
- By default, your internet traffic is either routed through an upstream proxy chosen by the administrator (so websites will see the proxy's address rather than this network's address) or sent direct, depending on how the administrator has configured this hotspot.
- Devices and services on the administrator's internal network (printers, file shares, internal tools, etc., within the configured bypass range) remain directly reachable, as if you were on that network normally.
- The administrator may also route specific websites differently from everything else (e.g. sending a particular streaming or banking site direct instead of through the proxy) — you do not need to do anything for this; it happens automatically based on which site you visit.

To confirm the proxy is active, visit any "what is my IP" site from a connected device and check that the address shown matches the upstream proxy's address, not your own.

■ **NOTE:** If a page fails to load or is very slow, it may be the upstream proxy itself being unreachable or overloaded — report this to the administrator rather than trying to change any settings on the hotspot, which is managed centrally.

7. Security Notes

• The admin web panel runs as an unprivileged, dedicated system account (proxyadmin-svc), never as root. It can only perform the exact, narrowly-scoped root actions listed in Section 4.2 Step 8 — nothing else — via sudo.

• Logging into the panel uses your real Linux account password, checked via PAM through a small root-owned helper (required because Linux's standard unix_chkpwd utility refuses to check another user's password from an unprivileged process). The password is passed to that helper over stdin, never as a command-line argument, so it never appears in `ps` output or in sudo's own logs.

• The panel is served over HTTPS with a self-signed certificate specifically so that Linux login credentials are not sent in clear text on the network — accept the browser warning, it is expected for a self-signed cert, but do not disable certificate checking generally on your own devices for other sites.

• The panel listens only on the wired/management interface's address, never on the hotspot's own Wi-Fi subnet. Devices on the hotspot can still connect to that address, as they can to SSH on the box: there is no inbound filter on the Wi-Fi side, so both rely on the Linux login. Use strong passwords.

• The upstream proxy's credentials and the hotspot passphrase are stored in plain text in root-readable configuration files on the box (`/etc/redsocks.conf`, the NetworkManager connection profile) — normal practice for this class of appliance, but worth knowing if the box's disk is ever removed or imaged.

• The DNS pattern-router runs as its own unprivileged account with only the CAP_NET_ADMIN Linux capability (granted via systemd, not sudo) — enough to manage ipsets and nothing else. It is on the hot path of every DNS query for the hotspot, so it is designed to fail open: any internal error just forwards the query unmatched rather than blocking DNS resolution, and it restarts automatically on crash.

• Every saved proxy profile's redsocks port listens on 0.0.0.0 (all interfaces), not just loopback — unchanged from the original single-proxy design, but worth knowing now that several profiles with different credentials can exist at once: anything able to reach the box directly on the wired/management side could, in principle, connect straight to a profile's port and relay through it, bypassing the hotspot entirely. This is only reachable from your own internal network, not the internet, given the existing firewall and NAT setup.

■ **NOTE:** The panel has no login rate-limiting or lockout. Consider adding fail2ban (or an equivalent) watching its access log if the box is reachable from an untrusted network segment.

8. Troubleshooting Reference

Issues encountered building and hardening the reference deployment, kept here so they are recognised quickly if they recur:

Symptom	Cause	Fix
The hotspot never starts (wlan0 shows "unavailable")	Wi-Fi country (regulatory domain) not set	Set the country in raspi-config, reboot, then unblock wifi, add
Static IP change on the wired interface silently does not take effect	A stored NetworkManager profile still owns the interface	Disable the conflicting profile, then `nmcli connect
Admin panel login always rejected, even with the correct password	unix_chkpwd (used by PAM) refuses to verify another user's password	Run the actual PAM check in a small root-owned helper via a
Web panel shows literal "{ip}", "{port}" etc. instead of real values	Template engine uses Python-style `{}` placeholders, not Jinja2 `{{ }}` syntax	Use Jinja2 throughout render_template_string c
Panel shows the redsocks local proxy port (0) instead of the real one	Config parser matched the wrong `port` line inside the `local_ip = ...; port = ...;` redsocks block	Anchor the regex on the start of the line (`^\s*port`) with the multiline flag
A sudo-invoked helper command fails with "command not found"	Minimal sudo secure_path omits /usr/sbin, where ipset and iptables live	Call them by absolute path (/usr/sbin/ipset, /usr/sbin/iptables)
Connected-devices list is empty despite a device having a lease	dnsmasq's lease file is in a root-only directory (/var/lib/NetworkManager)	Grant the panel read access to only that one file via a scoped sudoers
Saving the very first proxy profile fails on a fresh install	The hotspot-proxy-admin account does not exist yet, and the helper script cannot be owned by a non-existent user	Pre-create the dedicated system account before running the
A confirm() dialog on a button silently never appears, and the action just happens	A literal `\n` was used inside a plain (non-f, non-raw) Python string, which produces a real newline in the rendered	Avoid putting the same quoted string literal in both Python and

A pattern rule with a "proxy" action fails to save	REDIRECT needs TCP, UDP, SCTP or DCCP with a port specification	Choose a specific protocol, or change the rule's action to REDIRECT from just that one
A device briefly gets unrestricted, un-proxied internet access	The firewall rebuild helper flushes the REDSOCKS chain before re-adding its rules, and traffic arriving in that window	Treat any device-related failures as transient — re-running the rebuild helper
Traffic that should go through a specific proxy goes out the default instead, with no error shown	The pattern's proxy profile was deleted or disabled after the rule was created	Edit the rule and choose a valid, enabled proxy profile
`ipset list` accumulates sets no longer used by any rule	Deleting a pattern rule removes its firewall reference but leaves the ipset in the kernel	Harmless and invisible in normal use — the rebuild helper flushes and recreates ipsets on every rebuild
After a reboot, ALL hotspot traffic goes out directly, bypassing the proxy	The boot-time firewall rebuild service failed, so the REDSOCKS chain was never populated	Check `systemctl status rasp-proxy-firewall-init` and `journalctl -u rasp-proxy-firewall-init` for errors

■ **NOTE:** On Raspberry Pi OS Legacy/Bullseye (dhcpcd + hostapd + dnsmasq instead of NetworkManager), the static IP goes in /etc/dhcpcd.conf, the access point is configured via /etc/hostapd/hostapd.conf and /etc/dnsmasq.conf, and manual iptables MASQUERADE/FORWARD rules are required since there is no NetworkManager "shared" method to provide them automatically. The redsocks and firewall-bypass steps in Section 4.2 are unchanged either way.

■ **WARNING:** DNS lookups are not intercepted by redsocks (it only handles TCP), so they leave via normal routing rather than through the proxy — a device's DNS queries can reveal its browsing targets even while its actual connections are proxied. Treat the proxy as traffic-routing, not full anonymisation.

9. Reference

Item	Location
redsocks config (one block per saved profile, regenerated on every change)	/etc/redsocks.conf
Saved proxy profiles, each with its own permanent local port (from 20000)	/var/lib/rasp-proxy-admin/proxies.json
Internal bypass CIDR + global default (direct, or a profile), saved routing rules in priority order — the single source of truth the firewall is rebuilt	/var/lib/rasp-proxy-admin/settings.json
Admin web app	/opt/rasp-proxy-admin/app.py
DNS pattern-router daemon	/opt/rasp-proxy-admin/dns_router.py
This guide (served by the panel's Help page)	/opt/rasp-proxy-admin/Wi-Fi-Proxy-Hotspot-Guide.pdf
PAM check helper	/usr/local/sbin/rasp-proxy-pamcheck
Firewall rebuild helper (bypass CIDR + pattern rules + default)	/usr/local/sbin/rasp-proxy-rebuild-firewall
PAM service definition	/etc/pam.d/rasp-proxy-admin
TLS certificate/key	/etc/rasp-proxy-admin/tls/
Sudoers rule	/etc/sudoers.d/proxyadmin-svc
Admin panel systemd unit	/etc/systemd/system/rasp-proxy-admin.service
DNS router systemd unit	/etc/systemd/system/rasp-proxy-dns-router.service
Boot-time firewall rebuild systemd unit	/etc/systemd/system/rasp-proxy-firewall-init.service
dnsmasq forwarding to the DNS router	/etc/NetworkManager/dnsmasq-shared.d/pattern-routing.conf
Per-rule IP sets (tagged by the DNS router)	ipset, named wpx_<rule id>
DHCP lease file (hotspot side)	/var/lib/NetworkManager/dnsmasq-<ap-iface>.leases
Firewall chain	iptables nat table, chain REDSOCKS
Per-device traffic-accounting helper (adds/removes one counter rule per device)	/usr/local/sbin/rasp-proxy-traffic-sync

Per-device traffic counters (upload/download; not persisted across reboots by design)	iptables mangle table, chains WPXACCT_UP / WPXACCT_DOWN
Every known device ever seen (MAC, hostname, first/last seen, blocked)	/var/lib/rasp-proxy-admin/devices.json
Hotspot on/off state (survives reboot) and per-device block list	Gateway-AP connection's connection.autoconnect and 802-11-wireless.mac-address-denylist pr
Shared modules (rule matching, proxy check, DNS-over-HTTPS, proxy-aware urllib opener, e-mail sending)	/opt/rasp-proxy-admin/proxy_check.py, proxy_doh.py, mailer.py
DNS Lookups snapshot (RAM only, cleared on restart)	/run/rasp-proxy-dns/lookups.json
Health check helper, and its every-2-minutes timer	/usr/local/sbin/rasp-proxy-health; /etc/systemd/system/rasp-proxy-health.service and .timer
Latest health results (read by the Overview page; RAM only)	/run/rasp-proxy-health/status.json
Health alert state: debounce counters, unsent alert e-mails, last-known proxy exit IP	/var/lib/rasp-proxy-admin/health_state.json (root, 0600)
Email alert settings, including the SMTP password	/var/lib/rasp-proxy-admin/alerts.json (0600)

10. Rollback / Uninstall

From the package folder, as root:

```
sudo ./uninstall.sh
```

This removes the admin panel, the DNS pattern-router, both service accounts and the sudoers rule, the redsocks config, the firewall rules and any pattern-rule ipsets, and the hotspot connection. It deliberately leaves the wired interface's static IP and the installed packages (redsocks, iptables-persistent, python3-flask, python3-pam, gunicorn, iw, python3-dnspython, python3-qrcode, ipset, dnsmasq-base, openssl) in place, as well as a patched /usr/sbin/redsocks if Step 4b succeeded (reinstall the redsocks package to restore the stock binary); revert those manually if a full teardown is needed.

11. License, Credits and Disclaimer

License

Wi-Fi Proxy Hotspot is released under the **MIT License**, Copyright (c) 2026 Gerry Dassi. The full license text is in the LICENSE file of the repository and on the admin panel's Help page.

Disclaimer

> ■ **WARNING:** This software is provided "as is", without warranty of any kind. It is a personal project: it is not regularly maintained and no support is guaranteed. You are responsible for complying with the laws of your country, the policies of the network you connect it to, and the terms of service of your proxy provider and of any services you reach through it (for example geo-restricted streaming services).

Credits

The project is built on, and would not exist without, these open-source projects:

Project	Used for	License
redsocks	Transparent TCP-to-proxy redirector (one instance, one block per proxy profile)	Apache-2.0
Flask	Admin panel web framework	BSD-3-Clause
gunicorn	HTTPS server running the admin panel	MIT
dnspython	DNS handling in the DNS pattern-router	ISC
python-qrcode	Wi-Fi join QR code	BSD
NetworkManager	Wired configuration, Wi-Fi access point, DHCP, NAT	GPL-2.0+

iptables / ipset	Firewall chains, traffic redirection, per-rule IP sets, traffic counters	GPL-2.0
ReportLab (documentation only)	Generating this PDF guide	BSD

The two redsocks performance patches in package/redsocks-patches/ (Step 4b) are modifications of redsocks and are distributed under redsocks' own Apache-2.0 license, not the MIT license covering the rest of the project.